

Scientific Visualization

Practical work

1. Objective

The goal of the project is to create an interactive exploratory visualization using Altair and Streamlit to analyze the Henley Passport Index. The data can be downloaded from the Tidy Tuesday webpage: <https://github.com/rfordatascience/tidytuesday/blob/main/data/2025/2025-09-09/readme.md>.

The purpose of the Henley Passport Index is to capture the number of countries to which travelers in possession of each passport in the world may enter to the US visa free.

The purpose of this practical work is to create a series of interactive visualizations that analyze the evolution of visa-free access to the US. To do so, you need to create visualizations that answer the following questions:

- **Q1:** 1. Which continent has the most visa-free destinations? How does this vary when considering population size?
- **Q2:** Which countries have experienced the greatest changes as visa-free countries between 2006 and 2021?
- **Q3:** What was the impact of COVID-19 on visa-free mobility?
- **Q4:** Do countries that belong to certain global alliances, or have stronger economies, tend to enjoy greater visa-free access?

You need to clean the datasets if needed. The Tidy Tuesday page may give you some hints so clean the data, but you may add your own cleaning as well. Besides that, you will probably need to augment the dataset to incorporate related information.

For each **question**, a **chart** (or more than one) **needs to be designed and implemented**. You may tackle the problem in different ways: there is not a single solution. You also need to provide a **text of a maximum of 200 words per question** (and for the final visualization too) where you explain the design decisions and how these help the users to answer the questions properly. This should include aspects such as: what type of chart did you select and why, what were the different steps of design process you followed, what changes you applied to improve legibility, reduce clutter, distinguish elements, how would a person answer the question with your chart, what other alternatives did not work. Be concise. You do not need to erase the previous steps in design you followed, they are actually encouraged. But **only a final description per task** is necessary.

A **final visualization** is also required that **includes all questions**. In this case, you also have to explain, in a maximum of **200 words**, what changes were made to make charts consistent, aesthetically

pleasant, etc. The final visualization must be then **incorporated in a Streamlit** application (<https://streamlit.io/>). You have to teach yourselves on how to implement applications using Streamlit, which is very easy. You can start checking the initial videos of this playlist: <https://www.youtube.com/playlist?list=PLtqF5YXg7GLmCvTswG32NqQypOuYkPRUE>, or you can start with some of the videos by Fanilo Andrianasolo: <https://www.youtube.com/@andfanilo>. He has a subsection for starters with some interesting videos.

DATA

Data can be downloaded from the Tidy Tuesday webpage mentioned above. You may also use data from any other webpage you consider to be of sufficient quality; however, if you do so, you must describe it in your report. The data can be cleaned and enriched using OpenRefine or other tools.

DELIVERY INSTRUCTIONS

The work must be implemented individually. You have to provide the clean data. The person correcting the work should do no changes to the code to be able to correct. Therefore, you need to provide: **A functional Colab document that we can inspect. A Streamlit application with the proper datasets that executes with no changes:** just downloading your zip package, that should contain the data, should do the work.

The delivery must consist on a single ZIP file with a name that includes the author, that contains the datasets (raw and clean), the Colab file(s) (*ipnyb*), the Python Streamlit application, and optional extra documents if required. The ZIP, Streamlit, and Colab files must be named after the name of the author. Treat the Colab document as a report, include titles, boldfaces, etc., to make it easier to read.

The deadline for the delivery of this lab project is the 5th of November.

IMPORTANT REMARKS

The final grade will consider the number of variables included in the visualizations (these may include new calculated variables, such as aggregated data, averages, maxima, minima, etc.). Additionally, we will value the number of non-trivial tasks (adequately described in the documentation) that can be properly solved with your interactive visualization tool. In this sense, **adding extra tasks/questions is valued positively**.

Don't leave the project for the last day or do the minimum amount of work. In case of doubt, ask us whether the current work is enough or needs more effort.

THIS CHECKLIST SERVES AS GUIDANCE FOR YOUR DELIVERY

Global checklist:

- Name the file after the name(s) of the author(s).
- Include the name(s) of the authors also as the first line in your notebook.
- Include the clean data.
- Compress all files in a single zip (do not use RAR or other formats) file.

- Also include the names of the authors in the notebook (e.g., a text cell showing who authored the document) and in the Streamlit app (e.g., in an “About” option).
- Ensure the names of the data files inside the notebook correspond to the ones you deliver.
- Make a single delivery per group.
- Ensure you properly cleaned the data.
- Ensure the code executes without errors: last-minute changes may lead to typos.
- Ensure you include a step-by-step design process (does not need to include all minimal steps, if you prefer, but do not forget to include the discussion on why do you change something).
- Do not mix charts with other technologies, everything must be created using Altair.
- Ensure they do not include non-properly cleaned data (e.g., N/A or undefined fields).

Google Colab document. For every chart, you must consider:

- Color blindness (e.g., coding anything just with a red-green palette may be problematic).
- Check the consistency of colors across the whole visualization (same color, same meaning in different charts).
- Do not forget to add meaningful titles, labels, messages if necessary...
- When using colors with opacity different from zero, check the interactions with the other elements (are they visible?).
- Think whether you need to normalize values.

For the Streamlit vis:

- All questions should be solved in a single page (with not much scrolling, e.g., maximum double the available screen on a 15” laptop screen).
- The application should run with a simple “streamlit run <application_name>” in any computer, everything should be available in the same folder or addressed properly.
- Do not forget to include a final visualization that answers all the questions.
- Align things, be consistent. You can make use of both vertical and horizontal alignments to facilitate comparisons.
- Extra questions you answer must also go into the final vis.
- Ensure charts can be visually compared properly (consider changing scales, palettes...)
- Use space cleverly (put related things together and unrelated things far away).
- The higher the number of variables (and questions answered) included, the better.
- If you do not use all your data in your vis, ensure you have filtered it previously, don’t force Streamlit to execute with data it is not used.
- (Interactive part) Ensure the default values make sense (e.g., if including selections, check that the initial configuration of the chart has a default state that is meaningful).
- (Interactive part) Minimize the number of interactions required to solve the problems.